

Data Annotations in Entity Framework Core (EF Core)

In this article, I will discuss How to Configure Domain Classes with **Data Annotations in Entity Framework Core (EF Core)** with Examples. Please read our previous article discussing [Default Conventions in Entity Framework Core](#) with Examples.

What are Data Annotations in EF Core?

Data Annotations are attributes applied to domain classes and their properties to override the default conventions followed by Entity Framework Core (EF Core). They provide a simple way of configuring the database schema and enforcing validation rules directly within class definitions. Data Annotation Attributes are located in two primary namespaces. They are as follows:

- **System.ComponentModel.DataAnnotations:** For validation and metadata-related attributes.
- **System.ComponentModel.DataAnnotations.Schema:** For schema-related attributes that define database mappings.

Note: While Data Annotation Attributes are convenient for simple configurations, they are limited in functionality. For more advanced configurations, the EF Core's Fluent API is preferred, as it provides more configurations and flexibility.

Schema-Related Data Annotation Attributes in EF Core

These attributes impact the schema generation process in EF Core and reside in the **System.ComponentModel.DataAnnotations.Schema** namespace. They allow us to specify how EF Core should map domain models to database tables and columns.

Table Attribute:

Maps a class to a specific table in the database and allows us to configure the table name and schema.

```
[Table("Students", Schema = "Admin")]  
public class Student { }
```

Here,

- **Name:** Specifies the table name.
- **Schema:** Specifies the schema of the table.

Column Attribute:

Maps a property to a specific column in the database and allows configuration of the column name, order, and data type.

```
[Column("FirstName", Order=1, TypeName = "varchar(100)"]  
public string FirstName { get; set; }
```

Here,

- **Name:** Column name.
- **Order:** Column order in the table.
- **TypeName:** Database type of the column.

Note: The Order parameter is only effective in specific scenarios. EF Core does not guarantee column order in the table, so it's better to rely on the database migration script for ordering columns.

Index Attribute:

Adds an index to a property in the database. We can create only non-clustered indexes.

```
[Index]  
public string Email { get; set; }
```

Note: The Index attribute was introduced in EF Core 5.0 and must be applied to the class, not directly to the property.

ForeignKey Attribute:

Specifies the foreign key relationship between two entities explicitly.

```
[ForeignKey("TeacherId")]  
public Teacher Teacher { get; set; }
```

NotMapped Attribute:

Excludes a class or property from database mapping. Prevents EF Core from generating columns for the property.

```
[NotMapped]  
public string TemporaryData { get; set; }
```

Note: This attribute is ideal for properties used only in business logic but not intended for database storage.

InverseProperty Attribute:

Defines the inverse of a navigation property to clarify relationships when EF Core cannot determine them automatically, such as when multiple relationships exist between entities.

```
[InverseProperty("OnlineTeacher")]  
public ICollection<Course>? OnlineCourses { get; set; }  
[InverseProperty("OfflineTeacher")]  
public ICollection<Course>? OfflineCourses { get; set; }
```

DatabaseGenerated Attribute:

Specifies how the value of a property is generated in the database. Options include None, Identity, and Computed.

```
[DatabaseGenerated(DatabaseGeneratedOption.Identity)]  
public int Id { get; set; }
```

Options include:

- **None:** The database does not generate values; the application sets the value.
- **Identity:** The database generates values when new rows are inserted (e.g., auto-incrementing primary keys).
- **Computed:** The database generates values when rows are inserted or updated (e.g., computed columns).

Validation-Related Data Annotation Attributes

These attributes belong to the `System.ComponentModel.DataAnnotations` namespace, and are used to enforce validation rules on domain class properties before the data is sent to the database.

Key Attribute:

Marks a property as the primary key for the entity.

```
[Key]  
public int StudentId { get; set; }
```

Note: By convention, EF Core recognizes a property named `Id` or `[EntityName]Id` as the primary key. Use the `[Key]` attribute when the primary key property does not follow these naming conventions.

Required Attribute:

Ensures that the property cannot have a null value. This results in a **NOT NULL** constraint in the database.

```
[Required]  
public string LastName { get; set; }
```

Note: In EF Core 5.0 and above, when using nullable reference types (NRTs), properties declared as non-nullable (e.g., `string LastName`) are automatically treated as required. The `[Required]` attribute is still useful for enforcing validation in the application layer.

MaxLength/MinLength Attributes:

MaxLength: Specifies the maximum allowed length of a string or array property. It affects both validation and database schema (e.g., `varchar length`).

```
[MaxLength(50)]  
public string FirstName { get; set; }
```

MinLength: Specifies the minimum length for validation purposes but does not affect the database schema.

```
[MinLength(2)]  
public string FirstName { get; set; }
```

StringLength Attribute:

Specifies both the minimum and maximum length for string properties. The maximum length affects the database schema, while the minimum length is used for validation.

```
[StringLength(100, MinimumLength = 5)]  
public string Name { get; set; }
```

ConcurrencyCheck Attribute:

Marks a property to be used in concurrency control during updates. EF Core will include this property in the `WHERE` clause during `UPDATE` or `DELETE` operations to detect concurrency conflicts.

```
[ConcurrencyCheck]  
public string RowVersion { get; set; }
```

Timestamp Attribute:

This attribute is used for optimistic concurrency control. EF Core automatically manages the value, ensuring that updates do not overwrite changes from other transactions.

```
[Timestamp]  
public byte[] RowVersion { get; set; }
```

Note: The [Timestamp] attribute is equivalent to applying [ConcurrencyCheck] and configuring the property as a database-generated column.

Range Attribute:

Specifies a range of allowed values for numerical properties.

```
[Range(18, 65)]  
public int Age { get; set; }
```

Note: The [Range] attribute is used for validation purposes and does not affect the database schema.

EmailAddress, Phone, URL, etc.:

Used for specific validation patterns, such as verifying valid email addresses, phone numbers, or URLs.

```
[EmailAddress]  
public string Email { get; set; }
```

RegularExpression Attribute:

Validates that the property value matches a specified regular expression pattern.

```
[RegularExpression(@"\d{5}")]  
public string PostalCode { get; set; }
```

Fluent API vs. Data Annotations in Entity Framework Core:

- Data Annotations are easier to implement for simple configurations and validation but have limited capabilities.
- Fluent API provides much more flexibility and is the preferred method for advanced configurations like relationships, composite keys, table mappings, data seeding, global query filters, Shadow Properties, etc.

Note: If a conflict exists between Fluent API and Data Annotations, the Fluent API configuration takes precedence.

Combining Fluent API and Data Annotations in EF Core:

Using both Data Annotations and Fluent API together in the same project is also possible. Data Annotations handle simpler cases, while Fluent API is used for more complex configurations.

In the next article, I will discuss the [Table Data Annotation Attribute in EF Core](#) with Examples. In this article, I explain the data annotation attributes in Entity Framework Core.